

Mot de passe oublié — guide d'intégration mobile

Guide pour l'écran « mot de passe oublié » et son bouton **Renvoyer le code**. Toutes les réponses ci-dessous sont issues d'appels réels sur l'environnement de développement.

Référence : issue #243, PR #250.

1. Ce qui change pour l'application

Un nouvel endpoint, `POST /auth/resent-reset-code`, permet de **régénérer** le code de vérification. Jusqu'ici l'app n'avait que `forgot-password`, sans règle de cadence ni retour exploitable : impossible de savoir combien de temps griser le bouton de renvoi.

Trois points à retenir :

- **Chaque envoi produit un nouveau code et invalide le précédent.** Si

l'utilisateur demande deux codes, seul le dernier email fonctionne. L'email de renvoi le dit explicitement, mais l'écran doit le refléter aussi.

- **La réponse est toujours 201**, avec le même corps, que l'email existe ou

non, que la cadence soit respectée ou non. C'est délibéré : un statut variable permettrait de deviner quelles adresses ont un compte. **N'essayez pas d'en déduire quoi que ce soit.**

- **Le minuteur est piloté par le client**, à partir de `cooldown_seconds`

renvoyé par le serveur. Ne le codez pas en dur : la valeur peut changer.

Authentification. Ces trois endpoints sont **publics** — pas de `Authorization`.

Statut de succès : **201**, pas **200**. C'est le statut que renvoient déjà `forgot-password` et `reset-password` en production ; `resent-reset-code` s'aligne dessus. Si votre client teste `statusCode == 200`, il échouera — testez la plage `2xx`, ou **201** explicitement.

Scope pays. Contrairement au reste de l'API, **pas de `?country=` ni de champ `pays`**. L'authentification est volontairement inter-pays : un compte fonctionne quel que soit le pays sélectionné.

2. Le parcours

```
Écran « Mot de passe oublié » —
saisie de l'email
└─ POST /auth/forgot-password

      ↓ toujours 201
Écran « Saisie du code » —
6 chiffres + nouveau mot de passe
bouton « Renvoyer » grisé pendant cooldown_seconds
└─ POST /auth/resent-reset-code (régénère)
└─ POST /auth/reset-password (valide)

      ↓ 201
retour à l'écran de connexion
```

3. Demander un code — `POST /auth/forgot-password`

```
POST /auth/forgot-password
Content-Type: application/json

{ "email": "utilisateur@example.com" }
```

Réponse — 201

```
{
  "message": "Si l'email existe, un code a été envoyé",
  "cooldown_seconds": 60,
  "expires_in_seconds": 900
}
```

champ	sens
message	texte destiné à l'utilisateur, affichable tel quel
cooldown_seconds	secondes avant que « Renvoyer » redevienne actif
expires_in_seconds	durée de vie du code (900 s = 15 min)

L'utilisateur reçoit un email contenant un code à **6 chiffres**.

4. Renvoyer le code — `POST /auth/resent-reset-code`

Même corps, même réponse. C'est le bouton « **Renvoyer le code** ».

```
POST /auth/resent-reset-code
Content-Type: application/json

{ "email": "utilisateur@example.com" }
```

Réponse — 201

```
{
  "message": "Si l'email existe, un code a été envoyé",
  "cooldown_seconds": 60,
  "expires_in_seconds": 900
}
```

Ce que le serveur fait vraiment

La réponse ne le dit pas — c'est voulu — mais côté serveur :

situation	effet réel
cadence respectée, moins de 5 envois	nouveau code émis, email envoyé, l'ancien code ne vaut plus
moins de 60 s depuis le dernier envoi	aucun email , le code en cours reste valable
6 ^e envoi du cycle	le code est invalidé , il faut repartir de <code>forgot-password</code>
email inconnu	rien

Conséquence pratique pour l'app : après un renvoi, considérez que le code précédemment saisi n'est plus bon. Videz le champ de saisie et invitez l'utilisateur à prendre le code du **dernier** email reçu.

Les limites en vigueur

règle	valeur
délai entre deux envois	60 s
envois par cycle	5
tentatives de code	5
durée de vie du code	15 min

Un « cycle » démarre au premier `forgot-password` et se termine à la réinitialisation réussie, à l'expiration du code, ou à l'épuisement des envois ou des tentatives.

Ne codez aucune de ces valeurs en dur, sauf celles que l'API vous renvoie. Les plafonds (5 envois, 5 tentatives) ne sont pas exposés : traitez-les comme un motif d'échec générique, pas comme un compteur à afficher.

5. Valider le code — `POST /auth/reset-password`

```
POST /auth/reset-password
Content-Type: application/json

{
  "email": "utilisateur@example.com",
  "code": "417620",
  "nouveau_mot_de_passe": "NouveauMotDePasse123!"
}
```

Réponse — 201

```
{ "message": "Mot de passe réinitialisé avec succès" }
```

Toutes les sessions ouvertes sont fermées. Le `refresh_token` détenu par l'app — sur ce téléphone comme sur les autres — est révoqué. L'utilisateur doit se reconnecter avec son nouveau mot de passe ; ne tentez pas de rafraîchir le jeton après un succès, il répondra `401`.

Erreurs

401 — code refusé

```
{ "message": "Code invalide ou expiré", "error": "Unauthorized", "statusCode": 401 }
```

Un **message unique** couvre tous les cas : code faux, code expiré, code déjà utilisé, plafond de tentatives atteint, compte inconnu. Là encore, c'est délibéré : distinguer les motifs renseignerait un attaquant autant que l'utilisateur. Affichez le message tel quel.

Attention : après **5 codes erronés**, le code est invalidé — le **bon** code sera lui aussi refusé. La sortie est de repasser par « Renvoyer le code », qui reste soumis à la cadence de 60 s.

400 — validation du corps

```
{ "statusCode": 400, "message": ["Le mot de passe doit contenir au moins 6 caractères"] }
```

```
{ "statusCode": 400, "message": ["L'email doit être une adresse email valide"] }
```

`message` est un **tableau** sur les 400 et une **chaîne** sur les 401 — voir le helper `messageErreur` en section 7.

6. Ce que l'écran doit faire

Le minuteur. Démarrez-le sur la réponse de `forgot-password` et sur celle de `resend-reset-code`, à partir de `cooldown_seconds`. Le bouton reste grisé jusqu'à la fin. Le serveur absorbe silencieusement les demandes anticipées : sans minuteur côté app, l'utilisateur clique, ne reçoit rien, et vous n'avez aucun signal à lui donner.

Persistez le minuteur. S'il ne vit qu'en mémoire, quitter l'écran ou mettre l'app en arrière-plan le remet à zéro et rouvre un bouton que le serveur ignorera. Stockez l'horodatage de fin.

Videz le champ code après un renvoi. L'ancien code ne vaut plus.

N'affichez pas de compteur de tentatives restantes. L'API ne l'expose pas, et l'inventer côté client donnerait un chiffre faux dès que l'utilisateur reprend le parcours sur un autre appareil.

Ne déduisez rien de la réponse. Elle est identique pour un email inconnu. L'écran suivant s'ouvre toujours ; c'est l'absence d'email qui informe l'utilisateur, pas l'API.

7. Exemple Flutter

```
class MotDePasseOublieApi {
  MotDePasseOublieApi(this._client, {required this.baseUrl});

  final http.Client _client;
  final String baseUrl;

  /// Premier envoi. Renvoie la durée de grisage du bouton « Renvoyer ».
  Future<Duration> demanderCode(String email) => _envoyer('forgot-password', email);

  /// Renvoi : régénère le code. Le code précédemment reçu ne vaut plus.
  Future<Duration> renvoyerCode(String email) => _envoyer('resend-reset-code', email);

  Future<Duration> _envoyer(String chemin, String email) async {
    final reponse = await _client.post(
      Uri.parse('$baseUrl/auth/$chemin'),
      headers: const {'Content-Type': 'application/json'},
      body: jsonEncode({'email': email}),
    );

    final corps = jsonDecode(reponse.body) as Map<String, dynamic>;
    // Succès = 201 sur ces routes. Tester la plage 2xx plutôt qu'une valeur
    // exacte évite de recasser si le statut évolue.
    if (reponse.statusCode ~/ 100 != 2) throw ApiException(messageErreur(corps));

    // Valeur serveur : ne pas la coder en dur, elle peut changer.
    return Duration(seconds: corps['cooldown_seconds'] as int? ?? 60);
  }

  Future<void> reinitialiser({
    required String email,
    required String code,
    required String nouveauMotDePasse,
  }) async {
    final reponse = await _client.post(
      Uri.parse('$baseUrl/auth/reset-password'),
      headers: const {'Content-Type': 'application/json'},
      body: jsonEncode({
        'email': email,
        'code': code,
        'nouveau_mot_de_passe': nouveauMotDePasse,
      }),
    );

    if (reponse.statusCode ~/ 100 != 2) {
      throw ApiException(messageErreur(jsonDecode(reponse.body) as Map<String, dynamic>));
    }
    // Succès : toutes les sessions sont révoquées. Purger les jetons stockés
    // et renvoyer l'utilisateur sur l'écran de connexion.
  }
}

/// `message` est un tableau sur les 400 (validation) et une chaîne sur les 401.
String messageErreur(Map<String, dynamic> corps) {
  final message = corps['message'];
  if (message is List && message.isNotEmpty) return message.first as String;
  if (message is String) return message;
  return 'Une erreur est survenue.';
}
```

Le minuteur de renvoi

```
class MinuteurRenvoi extends ChangeNotifieur {
    DateTime? _finCadence;

    /// À persister (SharedPreferences) : un minuteur en mémoire seule repart à
    /// zéro au changement d'écran et rouvre un bouton que le serveur ignorera.
    void demarrer(Duration cadence) {
        _finCadence = DateTime.now().add(cadence);
        notifyListeners();
    }

    Duration get restant {
        final fin = _finCadence;
        if (fin == null) return Duration.zero;
        final reste = fin.difference(DateTime.now());
        return reste.isNegative ? Duration.zero : reste;
    }

    bool get renvoiPossible => restant == Duration.zero;
}
```

Après un renvoi

```
Future<void> onRenvoyer() async {
    final cadence = await api.renvoyerCode(email);
    minuteur.demarrer(cadence);

    // Le code précédent est mort : ne pas laisser l'utilisateur valider l'ancien.
    champCode.clear();
    afficherInfo('Un nouveau code vous a été envoyé. Les codes précédents ne sont plus valables.');
```

8. Rappels

Pas de jeton, pas de `?country=`. Ces trois endpoints sont publics et inter-pays, contrairement au reste de l'API.

La réponse d'envoi est constante. Même corps, même statut, dans tous les cas. Aucune logique d'affichage ne doit en dépendre.

Un renvoi invalide le code précédent. C'est la principale source de confusion sur ce type d'écran : traitez-la explicitement dans l'interface.

Le 401 de `reset-password` a un message unique. Il couvre le code faux, expiré, déjà utilisé et le plafond de tentatives. Affichez-le tel quel ; ne tentez pas de deviner le motif.

Après un succès, tous les jetons sont révoqués — y compris ceux des autres appareils. Purgez le stockage local et renvoyez vers l'écran de connexion.